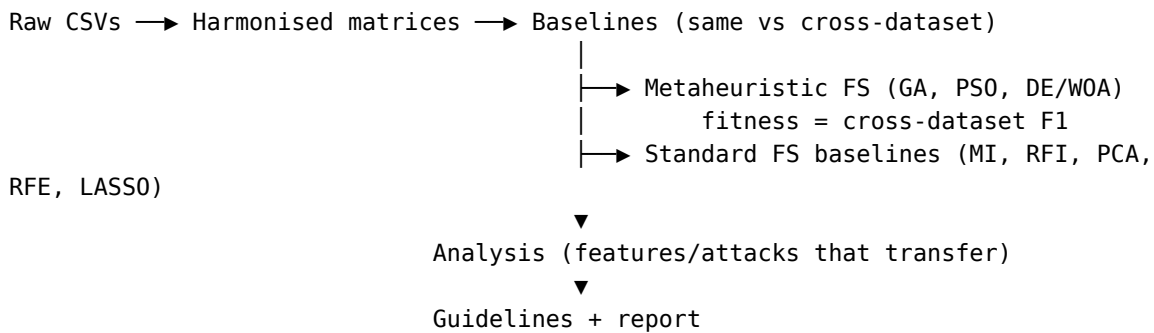


Methodology and Detailed Project Plan

Project: Evaluating and Improving Cross-Dataset Generalization of ML-Based Intrusion Detection Systems **Document:** 04 — Methodology & Plan **Status:** Prepared for mid-semester review

1. Overview

The project proceeds in seven phases. Phases 1–3 establish the baseline (how bad is the generalisation gap?); Phases 4–5 test the core hypothesis (does transfer-oriented metaheuristic feature selection narrow it?); Phases 6–7 interpret and consolidate. A single Python codebase (`src/`) implements all phases so every number in the reports is reproducible.



2. Environment & Repository Layout

- **Environment:** Python 3.11–3.13 virtual environment (`.venv`); dependencies `pandas`, `numpy`, `scikit-learn`, `imbalanced-learn`, `matplotlib`, `seaborn`, `joblib`, `xgboost`, `lightgbm`, `deap` (GA), `pyswarms` (PSO) or hand-rolled PSO/DE.
- **Repository:**
 - `datasets/` — raw CSVs (read-only)
 - `src/` — pipeline scripts (cleaning, harmonisation, models, optimisers, evaluation)
 - `reports/` — this document set + figures/
 - `results/` — cached matrices (`.parquet`), metrics (`.csv`), model artefacts
 - `notebooks/` — exploratory analyses

3. Phase 1 — Data Cleaning & Preprocessing

1. **Load** each dataset with whitespace-stripped headers; drop the duplicated CIC-IDS-2017 column `Fwd Header Length.1`.
2. **Fix non-finite values** in CIC-IDS-2017: replace Infinity in `Flow Bytes/s` / `Flow Packets/s` (and any other non-finite cells) with NaN, then impute (median) — or drop the affected rows (1,358 NaN + 4,376 Inf of ~2.83M rows; dropping is safe and clean).
3. **Normalise labels:** strip the en-dash encoding artefact in `Web Attack - *`; map to the binary target (benign vs attack) and the coarse taxonomy (Dataset Report §6.3).
4. **Remove leakage/identity features:** CIC-IDS-2017 `Destination Port`; TON_IoT `src_ip`, `dst_ip`, `src_port`, `dst_port`; UNSW-NB15 `id`.

5. **Handle categoricals** (TON_IoT primarily): label/one-hot low-cardinality columns (proto, service, conn_state); drop or frequency-cap very-high-cardinality text columns (http_uri, dns_query, http_user_agent, ssl_subject, ssl_issuer).
6. **Standardise numeric features** (zero mean / unit variance) using **per-dataset** scalers (never shared across datasets — a shared scaler would leak target statistics).
7. **Cache** harmonised DataFrames to .parquet for fast re-loading.

Deliverable: clean, harmonised matrices + a data_dictionary.md documenting every transform.

4. Phase 2 — Feature Harmonisation (shared subspace)

1. Implement the semantic mapping of Dataset Report §6.2 as a small declarative spec (JSON/YAML) mapping each shared semantic feature to its column name in each dataset.
2. Build the **shared numeric matrix** (~15–25 columns) plus **derived features** (e.g. bytes-per-packet, packets-per-second, fwd/rev byte ratio) that can be computed consistently across all three datasets.
3. Produce a feature-alignment report with a Venn/overlap table and a count of shared vs dataset-unique features.

Deliverable: harmoniser.py + shared-matrix builder; the alignment spec is a reusable contribution in itself.

5. Phase 3 — Baseline Benchmark (reproduce the “collapse”)

1. **Models:** Random Forest, XGBoost (or LightGBM), SVM (RBF), Logistic Regression, and a small MLP (e.g. 2×64 ReLU + dropout). Reasonable fixed hyper-parameters (tuned once on a small same-dataset fold, then frozen).
2. **Protocols:**
 - *Same-dataset:* stratified K-fold (K=5) within one dataset → expected high F1.
 - *Cross-dataset:* train on source, test on target, for all 6 ordered pairs of the 3 datasets (3×3 matrix, diagonal = same-dataset).
3. **Metrics:** balanced accuracy, macro-F1, per-class F1, and ROC-AUC (binary); report mean ± std over ≥5 random seeds.
4. **Output:** the full 3×3 source→target heatmaps (one per model/metric). This reproduces and extends [4], [5].

Hypothesis to confirm: same-dataset $\approx$ 0.97–0.99 F1; cross-dataset drops sharply, often near chance, and the drop is asymmetric across pairs.

6. Phase 4 — Metaheuristic Feature Selection for Cross-Dataset Robustness (core)

6.1 Representation

A candidate solution is a **binary mask** over the shared feature subspace: $x \in \{0, 1\}^d$, where $d \approx 15$ –25. (Optional extension: NSGA-II multi-objective version adding $|x|$ as a second objective to minimise the number of features.)

6.2 Fitness function (the key novelty)

The fitness of a mask is the **cross-dataset generalisation** of a fixed fast model trained on the selected features:

```
fitness(x) = mean over source→target pairs of ( F1 or balanced-accuracy  
          of RF/XGB trained on source[x] and evaluated on target[x] )
```

To prevent the optimiser from overfitting the target: - a **holdout source fold** is used for training, and each *target* is used only for scoring; - optionally, an inner cross-validation on the source is used as a proxy when a target is extremely small. - Fitness is averaged over the pairs (source, target) $\in \{\text{CIC} \rightarrow \text{UNSW}, \text{CIC} \rightarrow \text{TON}, \text{UNSW} \rightarrow \text{CIC}, \dots\}$ that include the current source; the search is run **per source dataset**, yielding the best transfer mask for each source.

6.3 Optimisers

Implement and compare at least three, each wrapped identically over the same mask representation and fitness:

1. **Genetic Algorithm (GA)** — binary tournament selection, uniform/two-point crossover, bit-flip mutation; population ~50, generations ~50 (using `deap`).
2. **Particle Swarm Optimization (PSO)** — binary PSO (sigmoid velocity) over the mask (using `pyswarms` or hand-rolled).
3. **Differential Evolution (DE)** or **Whale Optimization (WOA)** — binarised for feature selection.

Each run: ≥ 5 random seeds; record best mask, convergence curve, and fitness.

6.4 Model tuning (secondary objective)

After selecting features, lightly tune the fast model's key hyper-parameters (e.g. `RF n_estimators/max_depth`; `XGB eta/max_depth`) **using the same cross-dataset fitness**, to isolate the contribution of feature selection vs tuning.

Deliverable: `optimisers.py` + per-source best feature masks; convergence plots.

7. Phase 5 — Comparison against Standard FS Baselines

Run the *same* cross-dataset protocol using standard feature-selection methods, so the comparison is apples-to-apples:

Method	Type	Notes
All features (no FS)	baseline	upper reference on source fit
Mutual information (top-k)	filter	k swept
Random Forest feature importance (top-k)	embedded	k swept
PCA (top-k components)	transformation	not a subset, but a standard baseline
Recursive Feature Elimination (RFE)	wrapper	with a fast estimator
LASSO / L1 penalty	embedded	coefficients $\rightarrow$ subset
Correlation filter (drop	r	>0.9)

Each baseline selects a subset of the **same size** as the metaheuristic’s best mask (or is swept across k) and is evaluated under the identical cross-dataset protocol.

Primary comparison: does the GA/PSO/DE-selected subset achieve higher cross-dataset F1 than the best same-size standard-FS subset?

8. Phase 6 — Analysis & Characterisation

1. **Feature-transfer analysis:** which features are selected across seeds/sources (stability), and which are consistently *excluded* (dataset artefacts). Use permutation importance / SHAP on the shared subspace.
2. **Attack-class transfer:** per-class F1 confusion between source and target to find which attack categories transfer (e.g. DoS/scanning) and which do not (e.g. web/injection).
3. **Domain-shift visualisation:** t-SNE/UMAP of the shared subspace coloured by dataset to visualise the distribution gap before/after feature selection.
4. **Ablations:** (a) with vs without leakage features; (b) balanced vs natural class distribution (replicating the protocol-sensitivity finding of [6]); (c) number of selected features vs cross-dataset F1.

9. Phase 7 — Guidelines & Reporting

Distil Phases 3–6 into practical recommendations, e.g.: - which feature *families* are safe to rely on across networks (byte/packet/duration) vs fragile (ports, collection-window `ct_*`, protocol-specific `http_*/dns_*`); - a recommended feature subset and its expected cross-dataset performance envelope; - a recommendation on metaheuristic vs filter FS given a compute budget.

Consolidate into the final report and a short guideline sheet.

10. Timeline (one semester, 2-person team)

Weeks	Phase	Milestone
1–2	P1	Cleaned, cached matrices + data dictionary
3–4	P2	Shared feature subspace + alignment spec
5–6	P3	Baseline 3×3 cross-dataset matrix (reproduce [4]/[5])
7–9	P4	GA/PSO/DE transfer-oriented FS running
10–11	P5	Standard-FS comparison complete
12–13	P6	Feature/class transfer analysis + ablations
14	P7	Final report + guidelines

11. Success Criteria

- Reproduce the documented generalisation collapse on CIC-IDS-2017 / UNSW-NB15 / TON_IoT.
- Demonstrate a **statistically reliable** improvement in cross-dataset F1 for the metaheuristic-selected subset over (i) no-FS and (ii) the best standard-FS subset.
- Publish a reproducible codebase + the practical guidelines.

12. References

Numbered as in 02_literature_review.md; see [4], [5], [6], [9]–[13], [23].